

Vision-Only Autonomous Drone Inspection with SAM 3 Identification, Bearing-Only Triangulation, and Hybrid 3D Reconstruction in Simulation

Mohamed Gallai

*Department of Electrical and Computer Engineering
Binghamton University
Binghamton, NY, USA
mgallai@binghamton.edu*

Abstract—We present an autonomous target-inspection pipeline for a simulated unmanned aerial vehicle (UAV). Given only the natural-language name of an object, the system searches a populated room, identifies the target without fiducial markers or an object-position prior, orbits it twice at two altitudes, and produces four 3D outputs: target-only and full-scene 3D Gaussian Splatting reconstructions, a Depth Anything 3 (DA3) colored point cloud, and an Open3D Poisson triangle mesh. The pipeline is autonomous with respect to target discovery and mission execution, while the vehicle self-pose is provided by Gazebo ground truth in simulation. The system runs end-to-end in approximately 20 minutes per target on an 8GB consumer GPU. Object identification is performed with SAM 3, and object localization is solved using bearing-only triangulation across multi-view detections, with a 25° angular-spread gate that detects near-parallel-ray degeneracy. We evaluate the system by shuffling five test objects among five canonical positions at launch, so the drone has no SDF position prior and must visually rediscover each target. Across five targets and one moved-target test, four of six cases localize below 1 m, five of six are within 1.06 m, and the main outlier is the bench at 1.91 m. The work evolved away from the initial AprilTag and COLMAP proposal toward a SAM 3, DA3, and splatfacto hybrid, and the results suggest that an open-source ROS 2 and Gazebo stack can support modern foundation-vision models in a closed-loop simulation pipeline when the scene assets are chosen carefully.

Index Terms—autonomous drone, foundation models, SAM 3, 3D Gaussian Splatting, Depth Anything 3, ROS 2, Gazebo, bearing-only triangulation, 3D reconstruction, vision-language model

I. INTRODUCTION

Robotic 3D inspection of arbitrary, user-specified objects sits at the intersection of three traditionally separate problems: *recognition* (“find the fire hydrant”), *localization in the world* (“where is it in metric coordinates?”), and *geometry capture* (“produce a 3D model suitable for inspection”). Until recently, these were typically solved by attaching a fiducial marker to the target (e.g., AprilTag [1], [2]), running offline structure-from-motion such as COLMAP [3], [4] on a manually-piloted image set, and treating localization as a separate SLAM problem. Each step required calibration or human intervention; none generalized to a new object class without re-engineering.

In the last two years, three capability shifts have made this type of vision-centered simulation pipeline practical:

- **Open-vocabulary segmentation** via foundation models such as SAM [5], which segments any object given a point or box prompt, and SAM 3 [6], which extends this to short text concept prompts so that object categories can be detected and segmented without training a task-specific detector.
- **Photorealistic 3D representations** from photo sets: 3D Gaussian Splatting [7] supports high-quality, real-time novel-view rendering and is available through nerfstudio’s `splatfacto` implementation [8].
- **Robust monocular and multi-view depth estimation**: Depth Anything [9] provided strong monocular depth from a single image, and Depth Anything 3 (DA3) [10] extends the family to multi-view inputs by jointly predicting spatially consistent depth and refining camera extrinsics across the input set.

This work integrates these advances into a single autonomy pipeline running on top of ROS 2 Jazzy [11] and Gazebo Sim [12]. The drone receives only a text prompt (or a transcribed voice command, or an output of a local large language model), executes a structured search of an unknown room, visually identifies the requested object, orbits it twice at two altitudes, and emits four 3D output files. The localization step does not rely on any prior knowledge of object positions; instead, the drone fuses multi-view bearing measurements through a closed-form least-squares triangulation. To verify that the system genuinely relies on visual target discovery, we randomize object placement at every launch: five canonical positions are fixed, but the assignment of each physical object to a slot is shuffled, and that assignment is not exposed to the mission node.

A. Evolution from the original proposal

The original project proposal [24] specified an AprilTag fiducial marker for target detection and COLMAP for offline photogrammetric reconstruction. Both choices were dropped during implementation in favor of more general approaches. AprilTag forces a marker to be physically placed on the target, undermining the “vision-only autonomy” claim and limiting the system to a single known marker per mission.

SAM 3 replaces this marker requirement with open-vocabulary segmentation: the same pipeline can be launched with any of the five test-object names without changing the detector. Similarly, the structure-from-motion (SfM) stage of a COLMAP-style pipeline depends on stable feature matching, which is known to be challenging on the smooth, low-texture primitive surfaces present in our scene. The DA3 + splatfacto path was therefore a better fit for our simulator assets and for the desired combination of colored geometry and novel-view rendering.

B. Response to proposal feedback on photorealism

A reviewer noted that Gazebo Sim is not very photorealistic and may not support SAM 3’s image quality requirements. We acknowledge this concern and address it in two ways. First, we empirically verified that Gazebo’s `ogre2` renderer, when paired with carefully chosen asset materials and lighting, produces images on which SAM 3 detects the target reliably in our controlled scene (scores of 0.85 for the plant and 0.94 for the fire hydrant; see Section IX). Second, we replaced three Gazebo-Fuel community models that SAM 3 did *not* recognize with hand-built primitive geometries; after this change, the same object names became reliably segmentable. This does not make Gazebo generally photorealistic, but it shows that asset choice alone can make the simulator usable for this closed-loop perception experiment.

The reviewer also suggested SAM 3D from Meta as an alternative to classical photogrammetry. SAM 3D estimates a 3D shape from a single image and mask, which is well-suited to instant 3D-from-photograph applications but does not directly use the multi-view orbit collected by the drone. We instead use the multi-view DA3 model [10], which refines per-frame geometry and camera-pose consistency across the captured view set. The resulting point cloud is spatially coherent across all 48 selected frames; its metric accuracy is inherited from the simulator scale and camera-pose recording and should be validated quantitatively in future work. `splatfacto` adds a separate novel-view-rendering representation from the same image set.

C. Contributions

This paper presents the following contributions:

- 1) A full ROS 2 / Gazebo / SAM 3 / DA3 / `splatfacto` autonomy pipeline with end-to-end runtime under 20 minutes on an 8 GB consumer GPU.
- 2) A bearing-only triangulation algorithm for multi-view object localization, with a closed-form least-squares solution and a 25° angular-spread gate to detect and reject parallel-ray degeneracy. The gate triggers a fallback to a score-weighted spatial median.
- 3) A scene-randomization mechanism that tests whether the system is genuinely vision-driven: by shuffling five objects among five canonical slots at every launch and hiding the assignment from `mission_node`, we eliminate the possibility of an SDF position prior leaking into the pipeline.

- 4) An empirical evaluation across five targets and one moved-target test, with four of six localization errors below 1 m and five of six within 1.06 m, without any target-position prior.
- 5) A virtual-gimbal pitch correction during orbit that keeps the target near image-center across both orbit altitudes, preventing tracker drift to nearby distractors.

II. RELATED WORK

A. Visual fiducials and open-vocabulary segmentation

AprilTag [1] introduced a robust, low-cost fiducial marker system that has become a standard tool for autonomous robot localization. Subsequent work [2] improved detection efficiency. While reliable, fiducials require physical placement on the target and identify only known marker IDs. The Segment Anything Model (SAM) [5] eliminated the need for class-specific training: SAM produces high-quality segmentation masks for any object given a point or box prompt. Subsequent work extended segmentation pipelines toward text grounding and video tracking. SAM 3 [6] introduces promptable concept segmentation, in which short noun phrases, image exemplars, or both can be used to detect, segment, and track matching object instances. In this project, `sam3_detector` uses the SAM 3 semantic segmentation interface for image-frame target masks.

B. 3D reconstruction from images

Classical structure-from-motion pipelines such as COLMAP [3], [4] dominated 3D reconstruction for a decade but require well-textured surfaces and tend to fail on simulated assets with smooth materials. Neural radiance fields (NeRF) [15] introduced volumetric rendering for novel view synthesis but at very high training cost. 3D Gaussian Splatting [7] replaced volumetric integration with explicit anisotropic Gaussian primitives, achieving real-time rendering and dramatically faster training. The `nerfstudio` framework’s `splatfacto` [8] provides a turnkey 3DGS implementation which we use here.

For monocular depth estimation, the Depth Anything family [9] demonstrated that a vision transformer trained on 62 M images plus 1.5 M depth labels can predict high-quality depth on arbitrary scenes. Depth Anything 3 (DA3) [10], used in this work, extends the family to multi-view inputs and jointly refines per-frame depth and camera extrinsics across the input set, which is useful for fusing the orbit image set into a single colored point cloud.

C. UAV planning for reconstruction

The UAV photogrammetry literature has long emphasized that reconstruction quality depends on capture pattern. Zhang et al. [20] demonstrated that self-occlusion makes simple orbit patterns insufficient and motivated multi-altitude capture. Our two-ring orbit (low + high) follows this recommendation. Recent work on adaptive view planning [21] optimizes views to maximize information gain; we leave adaptive planning as future work and adopt a deterministic two-altitude orbit because it is sufficient for our object scale and orbit radius.

D. ROS 2 and Gazebo for UAV simulation

ROS 2 [11] is the de facto open-source robot middleware. Gazebo Sim [12] provides physics, sensors, and rendering for robot simulation. The `ros_gz_bridge` translates between Gazebo’s native transport and ROS 2 topics, enabling foundation-model-based perception code (which expects ROS topics) to consume simulated camera streams transparently. We selected Gazebo over alternatives such as Isaac Sim and AirSim for this specific project because Gazebo provides native ROS 2 integration, simple SDF/XML scene authoring, deterministic kinematic pose control, and reliable operation on the available 8 GB GPU. This choice was practical rather than universal: current Isaac Sim documentation lists 16 GB of VRAM as the minimum specification for the 5.1 workstation workflow and warns that lower-VRAM GPUs may be insufficient for complex rendering workloads [13], while the official Microsoft AirSim release page lists v1.8.1 (2022) as the most recent release [14].

III. SYSTEM ARCHITECTURE

The system comprises five primary ROS 2 nodes plus three auxiliary nodes (Fig. 1). `mission_node` is the state machine and motion planner, owning all decisions about where the drone goes next. `sam3_detector` hosts the SAM 3 model on GPU and publishes per-frame detection results. `image_capture` buffers the camera stream and saves frames plus poses to disk on trigger. `ros_gz_bridge` translates between Gazebo’s native transport and ROS 2. The auxiliaries are `scene_randomizer` (one-shot at startup, shuffles object positions), `flight_logger` (writes per-tick CSV telemetry), and `live_view` (a custom OpenCV window that overlays the SAM 3 mask on the live camera stream).

A. Why no SLAM?

The `mission_node` uses Gazebo’s ground-truth drone pose obtained via the bridged `/world/recon_world/set_pose` service. This means that the project evaluates autonomous target discovery, target localization, and image capture, but it does *not* evaluate vehicle state estimation. SLAM would be required for hardware deployment or for a simulation study focused on drone self-localization; here, using simulator ground truth keeps the experiment focused on the unknown target object’s position.

B. Why kinematic teleport, not velocity-controlled flight

The drone is driven via direct `SetEntityPose` writes every 50 ms (kinematic mode) rather than through `Twist` commands on `/cmd_vel` (velocity_kinematic mode), although both modes are implemented. The motivation is splat-training data quality. 3D Gaussian Splatting back-projects each captured image through its associated camera pose to compute reprojection error; if the pose recorded in `transforms.json` disagrees with the camera’s actual pose at capture time by even a few centimeters, the optimizer cannot fully reconcile the contradiction and the result can

become a visibly smeared splat with floater artifacts. In velocity-controlled flight, the closed-loop controller tracks its commanded waypoint with 5–10 cm of steady-state error from inertia, propeller dynamics, and integration noise, so the recorded pose and the actual capture pose can drift apart by exactly the wrong amount. Kinematic pose control eliminates this gap by construction: the recorded pose is the actual simulator pose because the planner writes it directly. Visually, the motion remains close to continuous because the per-tick position change is approximately 5 cm in search and 1 cm in orbit at 20 Hz, but the data written to disk is repeatable and pose-consistent. Velocity mode remains available for runs that demonstrate the autonomy logic under more realistic flight dynamics; for hardware deployment, it would be replaced with PX4 or ArduPilot velocity commands without changing the perception, planning, or reconstruction stages.

IV. VISION-ONLY OBJECT IDENTIFICATION AND LOCALIZATION

A. SAM 3 segmentation

The SAM 3 semantic segmentation interface is loaded onto the GPU at startup (approximately 2 GB of VRAM in our implementation) and runs at approximately 2 Hz on a 1280×720 camera frame downsampled to 644 px for inference. For every prompt (e.g., “potted plant”), SAM 3 returns a list of candidate masks with associated confidence scores. We select the mask with the highest confidence score rather than the largest area; this is a stronger signal of correct identification when multiple objects of similar shape are visible (e.g., a green trash bin near a green plant).

Fig. 2 shows a representative detection. The overlay is a live operator-facing diagnostic, not a separate algorithmic input: it lets the user confirm what the drone is seeing, which target SAM 3 has selected, the current mask and bounding box, the centroid used for ray back-projection, and the depth-based range estimate.

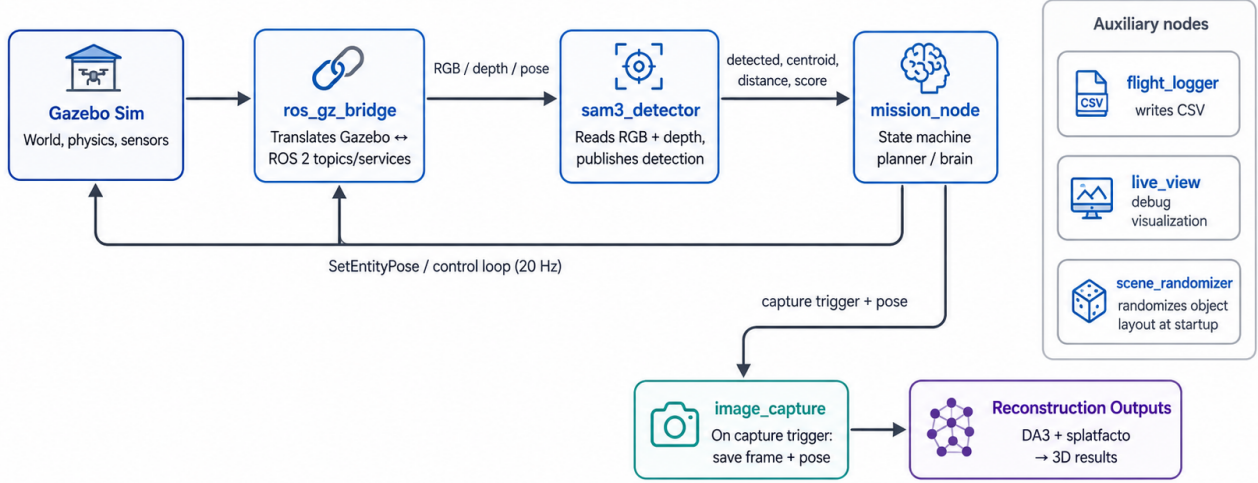
B. Distance via masked depth sampling

To convert a 2D mask into a 3D position, we sample the drone’s depth camera under the segmentation mask. A naive centroid sample is brittle on porous objects (e.g. plant foliage) because background depth bleeds through gaps. We mitigate this in three steps:

- 1) **Mask warp.** The RGB camera is 1280×720 and the depth camera 640×480 ; both share intrinsics and pose up to a focal-length scale. We apply a closed-form affine warp to project the mask into depth-image coordinates.
- 2) **Erosion.** The mask is eroded by a few pixels with OpenCV to remove silhouette-edge bleed-through onto the background.
- 3) **10th-percentile aggregation.** Among the surviving valid depth values, we take the 10th percentile rather than the median. This biases toward the closer surface, which is virtually always the actual object when the mask includes some background.

ROS 2 Drone Reconstruction System

Simplified data flow and node roles



Each component is decoupled: sensors, detection, planning, logging, and reconstruction communicate through ROS 2 topics/services.

Fig. 1: Simplified ROS2 data flow. Five primary nodes plus three auxiliaries (`flight_logger`, `live_view`, `scene_randomizer`) communicate via topics and services. The kinematic teleport loop runs at 20 Hz; SAM 3 inference runs at approximately 2 Hz. No SLAM is used: the drone uses Gazebo’s ground-truth vehicle pose, while the target position is estimated from vision-based detections.

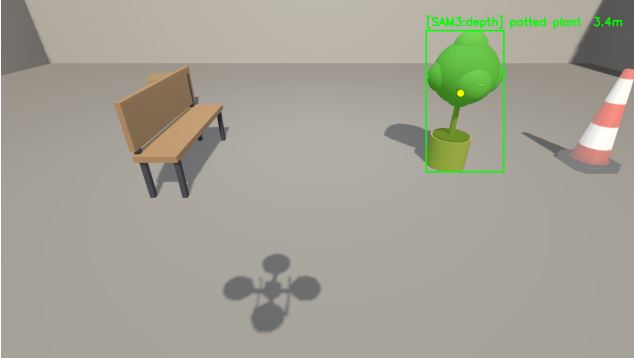


Fig. 2: SAM 3 detection example. The drone has just confirmed a `potted plant` target at score 0.85; the green overlay is the per-pixel mask, the rectangle is the bounding box, and the yellow dot is the centroid back-projected through the depth camera. The text label `[SAM3:depth] potted plant 2.0m` indicates the slant range computed by the masked-depth procedure.

C. Bearing-only triangulation

The estimated distance is noisy ($\sim 30\text{--}50$ cm RMS error per hit even after the masked sampling above). To avoid relying on it, we compute a 3D *bearing ray* from the drone’s pose and the mask centroid pixel. Each detection produces a tuple

$(\mathbf{o}_i, \mathbf{d}_i)$ of ray origin and unit direction in the world frame.

Across multiple drone positions, we solve for the world point $\mathbf{p}^*$ closest in the least-squares sense to all rays. Each ray contributes a 3×3 projection matrix $\mathbf{P}_i = \mathbf{I} - \mathbf{d}_i \mathbf{d}_i^\top$ that nullifies components along the ray direction. The constraint $\mathbf{P}_i(\mathbf{p}^* - \mathbf{o}_i) \approx \mathbf{0}$ holds approximately for each i , giving the closed-form normal equations

$$\left(\sum_i \mathbf{P}_i \right) \mathbf{p}^* = \sum_i \mathbf{P}_i \mathbf{o}_i \quad (1)$$

which we solve directly. Crucially, this estimator depends on *ray directions only*; the noisy depth measurement is used for fallback estimates and diagnostics but does not enter the normal equations. We validated the formulation on synthetic data: five rays from different vantages with 2° direction noise each produced a 33 cm RMS error to a target at $(3, 4, 0)$.

In summary, each successful SAM 3 detection provides a mask centroid in the image, which is back-projected into a unit bearing ray $\hat{\mathbf{d}}_i$ in the world frame at the camera origin $\mathbf{o}_i$. After five accepted detections from different drone poses, the target position $\hat{\mathbf{p}}^*$ is the point closest to all rays in the least-squares sense (1). An angular-spread check (next subsection) rejects near-parallel ray geometry and triggers a depth-projected median fallback.

D. Parallel-ray detection

The system in (1) is well-conditioned only when the rays span a meaningful angular range. For nearly-parallel rays (drone moving in a straight line, target at a stable image position), $\sum_i \mathbf{P}_i$ becomes singular and the least-squares solution drifts toward the centroid of the ray origins, which is wrong by exactly the triangulation baseline.

We detect this by computing the maximum pairwise angle between ray directions in the cluster:

$$\theta_{\max} = \max_{i \neq j} \arccos(\mathbf{d}_i \cdot \mathbf{d}_j) \quad (2)$$

If $\theta_{\max} < 25^\circ$, we reject the triangulation and fall back to the spatial median of the per-hit distance-projected XY estimates. The threshold was tuned empirically: in our tests, spreads below $\sim 20^\circ$ produced clearly diverged solutions, while spreads above $\sim 30^\circ$ produced sub-meter results. We use 25° as a conservative midpoint between these regimes.

E. Multi-view gate and spatial clustering

Without care, the 20Hz mission timer ingests the same SAM 3 detection multiple times before the next inference fires, producing zero-parallax duplicates. The multi-view gate requires the drone to have moved at least 30 cm or yawed at least 10° between consecutive accepted hits.

Because SAM 3 can occasionally misidentify objects of similar shape, we cluster hits spatially. A new hit joins the first existing cluster whose centroid lies within 2 m; otherwise, it spawns a new cluster. A cluster is confirmed once it accumulates five hits, has an average SAM 3 score of at least 0.45, and meets the 25° spread requirement. If the search exhausts without confirming any cluster, we instead select the cluster with the highest average SAM 3 score (with at least two hits) and triangulate that cluster.

V. MISSION PLANNING AND CONTROL

`mission_node` runs an eight-state machine at 20Hz: IDLE $\rightarrow$ SEARCH $\rightarrow$ APPROACH $\rightarrow$ ORBIT_LOW $\rightarrow$ CLIMB $\rightarrow$ ORBIT_HIGH $\rightarrow$ RETURN $\rightarrow$ LAND.

A. Search trajectory

The drone climbs to a search altitude of 1.8 m and executes a boustrophedon (lawnmower) sweep over four columns at $x \in \{-3, -1, +1, +2.5\}$ m, with y alternating between +4 and -4 m. Search speed is 1 m/s, intentionally slow so that SAM 3 produces multiple frames of detection per object as the drone passes nearby. The search waypoints are inside the room walls with margin; the trajectory is precomputed and not reactive, because there are no dynamic obstacles in the scene. Reactive planning would be the natural extension for outdoor or shared spaces.

B. Approach

When a cluster confirms, the drone heads in a straight line toward the locked target XY at constant altitude until it is within `orbit_radius` + 0.3 m horizontally. The 0.3 m hysteresis prevents oscillation at the boundary.

C. Orbit and virtual gimbal pitch

Both orbits are circular trajectories at the locked target XY:

$$\begin{aligned} x(\phi) &= t_x + R \cos \phi, \\ y(\phi) &= t_y + R \sin \phi, \\ \text{yaw}(\phi) &= \text{atan2}(t_y - y, t_x - x) \end{aligned} \quad (3)$$

where ϕ advances at 0.15 rad/s. The yaw line keeps the camera rigidly pointed at the target's horizontal position.

The drone's camera is mounted with a fixed 30° downward pitch in the SDF. At ORBIT_LOW (altitude 1.5 m, radius 1.5 m), this geometry naturally centers the target in the frame. At ORBIT_HIGH (altitude 3 m, same radius), however, the optical axis hits the ground at $3 / \tan(30^\circ) \approx 5.2$ m horizontally, far past the target. The target slides to the bottom edge of the frame, and the OpenCV tracker (used during orbit for speed) drifts to whatever stays centered (e.g. a nearby cone in our scene).

To address this without adding a mechanical gimbal, we introduce a virtual gimbal: the drone body is pitched forward by the geometric correction

$$\theta_{\text{pitch}} = \text{atan2}(z_{\text{drone}} - z_{\text{target}}, R) - \theta_{\text{cam}} \quad (4)$$

where $\theta_{\text{cam}} = 30^\circ$ is the static camera mount angle. At ORBIT_LOW the correction is approximately -2° (essentially level); at ORBIT_HIGH it is approximately $+27^\circ$ (nose-down). Because the drone is in kinematic mode, this is a quaternion-only change with no physics implications. The pitch resets to 0 on every non-orbit state transition.

D. Image capture

`image_capture` buffers the camera stream and the `/drone/pose` topic continuously. On every Bool pulse on `/mission/capture` (fired every 7.5° of orbit angle by `mission_node`), it writes the latest frame to disk and appends the camera pose to a `transforms.json` file in nerfstudio format and a `poses.json` file for DA3. With a 7.5° angular step, each orbit produces 48 captures, and the two altitudes together yield up to 96 image-pose pairs that are subsequently used as input to the reconstruction stage.

VI. 3D RECONSTRUCTION PIPELINE

On MISSION COMPLETE, `image_capture` launches the reconstruction pipeline as a detached subprocess so it survives mission shutdown. The pipeline produces four output files in `~/recon_output/exports/`.

A. DA3 colored point cloud

DA3 is invoked through its CLI (`da3 auto`), which: (1) runs depth inference per image, (2) jointly refines camera extrinsics across the input frames, and (3) fuses per-frame depth maps into a single colored 3D point cloud. We use the DA3-LARGE variant (~ 0.35 B parameters), which fits within our 8 GB GPU memory budget; the DA3NESTED-GIANT-LARGE variant (1.4 B parameters) exceeded the available 8 GB memory budget and produced out-of-memory errors in our testing. DA3's native output format

is GLB (binary glTF 2.0); we added a small trimesh-based conversion step to extract the embedded point cloud and write a Stanford PLY file (`scene_da3.ply`). Inference takes approximately 3 minutes.

B. Open3D Poisson surface reconstruction

The DA3 cloud is then passed through Open3D’s Poisson surface reconstruction [16], [17], which estimates per-point normals and fits an implicit surface to the point cloud. The result, written to `scene_mesh.ply`, is a continuous triangle mesh suitable for visual inspection and import into 3D modeling software. Because we do not compare the mesh to ground-truth geometry in this report, measurement accuracy should be treated as approximate. Poisson reconstruction takes approximately 30 seconds.

C. Splatfacto 3D Gaussian Splatting

splatfacto [8] trains a 3DGS model on the captured images and their poses for 30 000 iterations (approximately 10–15 minutes on our hardware). The trained model is exported via `ns-export gaussian-splat` twice: once unrestricted (`scene_splat.ply`, the whole room) and once restricted to a per-target axis-aligned bounding box defined in `targets.py` (`splat_pruned.ply`, the orbited object only).

D. Order of operations

DA3 runs before splatfacto. The reason is VRAM management: splatfacto’s nerfstudio viewer holds approximately 4 GB of VRAM after training and may remain alive for user inspection; if splatfacto ran first, DA3 (which needs roughly 3–4 GB at inference) could exceed the remaining headroom on the 8 GB GPU. With DA3 first, the DA3 subprocess fully exits and releases its memory before splatfacto initializes. Total wall-clock per target is approximately 15–18 minutes.

VII. USER INPUTS AND CONTROL MODALITIES

The system accepts target prompts through four interfaces, all of which converge on the same launch argument `target:=<name>`:

- 1) **Direct text** via the launch argument (e.g. `ros2 launch drone_recon scenel.launch.py target:="potted plant"`).
- 2) **Voice**: the `voice_mission` node records audio from the system microphone, transcribes it with faster-whisper (a quantized variant of Whisper [18]), parses the intent, and calls `execvp` on the same launch command.
- 3) **Graphical interface**: `voice_gui` (Fig. 3) provides a Tkinter window with a microphone button and a text-fallback field for users without microphone access.
- 4) **Free-form natural language**: `ai_mission` dispatches the input through Qwen 2.5 7B [19] hosted in Ollama. The LLM is presented with a tool schema for `start_inspection(target)` and `start_mapping()`; it returns a structured tool call that is converted into the launch argument vector. Qwen

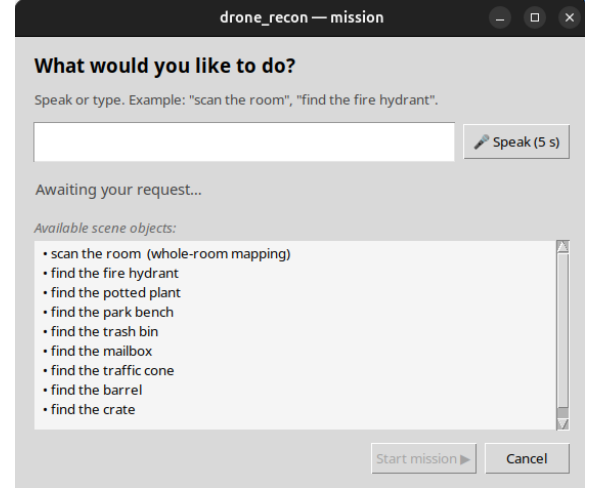


Fig. 3: The `voice_gui` Tkinter frontend, showing the text-and-voice entry box and a list of detected scene objects that the user may target.

is unloaded from GPU memory before the mission launches in order to free VRAM for SAM 3 and DA3.

VIII. SCENE RANDOMIZATION

To ensure every run is a genuine vision-based target-discovery test, we shuffle which physical object occupies each canonical slot before the drone takes off. The five canonical positions are fixed $((0, 0), (-3.5, 1.5), (-3.5, -1.5), (1.5, -2.0), \text{ and } (1.5, 2.0) \text{ m})$, but the assignment object $\rightarrow$ slot is random.

`scene_randomizer` is a one-shot ROS 2 node started 4s after the launch. It calls `SetEntityPose` once per object to teleport each model to its randomly-chosen canonical slot, then exits. A `seed:=N` parameter (default 0, meaning OS entropy) reproduces a given permutation for repeatable testing. Critically, `mission_node` hard-codes the `expected_target_xy` field to `None`, and there is no API by which the mission node can query an object’s position. The drone must therefore identify the named object visually for each run rather than reading its target position from the SDF file.

IX. RESULTS

A. Experimental setup

All experiments were performed on a single workstation with an NVIDIA mobile GPU with 8 GB of VRAM, an AMD Ryzen-class CPU, and 32 GB of system RAM, running Ubuntu 24.04 LTS. The robotic stack consists of ROS 2 Jazzy Jalisco and Gazebo Sim Harmonic (`gz sim 8.x`), bridged through `ros_gz_bridge`. The drone is a custom hand-authored X3 quadrotor model defined in SDF, equipped with one 1280×720 RGB camera at 15 Hz, one 640×480 depth camera at 10 Hz, a 360-degree GPU LiDAR with 16 vertical channels, and an IMU. Only the RGB and depth streams are used by the perception pipeline. The simulated room is 20 × 16 m with

TABLE I: Target Localization Accuracy — vision-based target discovery, no SDF target-position prior

Target	Actual (m)	Estimated (m)	Err.	Score
Plant (default)	(−3.50, 1.50)	(−3.66, 1.84)	0.36 m	0.85
Plant (moved)	(+2.00, −3.00)	(+1.56, −2.19)	0.92 m	0.91
Hydrant	(0.00, 0.00)	(+0.16, −0.34)	0.37 m	0.94
Trash bin	(+1.50, −2.00)	(+1.39, −2.18)	0.21 m	0.87
Mailbox	(+1.50, +2.00)	(+2.55, +2.15)	1.06 m	0.64
Bench	(−3.50, −1.50)	(−5.40, −1.33)	1.91 m	0.72

walls 3.5 m tall. The mission control loop runs at 20 Hz, and SAM 3 inference runs at approximately 2 Hz with stride 2. Each mission captures up to 48 images per ring across two altitudes (a low orbit at 1.5 m and a high orbit at 3.0 m); the orbit radius is defined per target in `targets.py` and is typically in the 1.5–2.0 m range. End-to-end runtime per target is approximately 20 minutes: ~5 minutes for flight and capture, ~3 minutes for DA3 inference, ~30 seconds for Open3D Poisson meshing, and ~10–15 minutes for splatfacto training (30 000 iterations).

The Gazebo world is hand-authored SDF, which is Gazebo’s plain-XML scene format. Off-the-shelf Gazebo Fuel community meshes were used for the fire hydrant and construction cones, while the room walls, floor, lighting, and four primitive replacement targets (bench, trash bin, mailbox, and potted plant) were authored manually. The drone itself is a custom primitive model: the design, including sensor placements, body proportions, gimbal location, and +-configured propellers, was specified by the author. Because the resulting drone block exceeded 200 lines of repetitive XML covering body links, motor housings, propellers, landing legs, and sensor declarations, a large language model was used to expedite conversion of the design into well-formed SDF. The output was then iteratively refined by hand to address issues such as a frame-edge occlusion artifact (resolved by relocating the camera below the body) and to tune materials for SAM 3 compatibility. The final world file is approximately 800 lines of SDF, roughly one quarter of which is the drone model.

B. Localization accuracy

Table I reports localization error for each of the five targets at default positions plus a moved-plant test (plant relocated from its default (−3.5, 1.5) to (+2.0, −3.0) prior to launch). All results are with the `expected_target_xy = None` configuration; the drone has zero advance knowledge of object positions.

Four of the six cases produced sub-meter localization error, and five of six were at or below 1.06 m, with the trash bin as the best case at 21 cm. The bench (1.91 m error) and mailbox (1.06 m error) are the weakest cases, and three factors compound their difficulty. First, both objects have a visually weaker silhouette than the others: the bench is a long, thin, low-profile slab on legs, and the mailbox is a small box on a thin post. SAM 3 confidence on these is correspondingly lower (0.72 and 0.64, respectively) compared with 0.85–0.94 for the more volumetrically prominent plant and hydrant. Second, both silhouettes change substantially between viewing angles:

the bench appears as a long rectangle from the side and a thin line end-on, while the mailbox changes apparent shape with parallax around the post. This makes the per-frame mask centroids less consistent and increases triangulation noise. Third, both objects are placed in regions of the room with strong distractors—the bench near the south wall corner and the mailbox near a traffic cone—which sometimes capture borderline mask candidates. These observations are consistent with the broader behavior of open-vocabulary segmentation: foundation models such as SAM 3 perform best on objects with iconic, easily distinguishable shapes from many view-points and degrade on flatter or more ambiguous geometry. Replacing the bench primitive with a more iconic mesh (e.g., one with visible wood grain and clearly rounded armrests) is the highest-value asset improvement identified from these tests.

The moved-plant case is the strongest evidence for genuine vision-only autonomy: with the plant relocated to a slot whose assignment was not pre-registered with the mission node, the drone still located it within 1 m using SAM 3 segmentation and bearing-only triangulation alone.

This evaluation is intentionally a small, controlled proof-of-concept: five visually distinct objects (a fire hydrant, a plant, a wooden bench, a gray trash bin, and a USPS-style mailbox) at five canonical positions, plus one moved-object stress test. This set is sufficient to demonstrate that the identification, triangulation, and orbit pipeline generalizes across object classes and across positional layouts the drone has not previously encountered, but it is not intended as a benchmark on the diversity of scene clutter. Scaling the evaluation to larger object sets is straightforward in software because SAM 3 is open-vocabulary and requires no per-object retraining; we leave such a benchmark to future work.

C. Reconstruction outputs

Fig. 4 shows representative reconstruction outputs across three of the four output formats. The pruned splat (Fig. 4a) gives a target-focused 3DGS export of the orbited fire hydrant. The full splat (Fig. 4b) includes the surrounding scene walls and lighting. The DA3 colored point cloud (Fig. 4c) provides a coherent colored geometry estimate that supports qualitative inspection and approximate measurement in simulator coordinates. The Poisson mesh (not shown for space) is a continuous triangle surface derived from the cloud in Fig. 4c. Reconstruction quality is evaluated qualitatively in this report; quantitative geometry metrics such as mesh-to-ground-truth error, Chamfer distance, PSNR, and SSIM are left for future evaluation.

D. Reconstruction evaluation methodology

Reconstruction outputs in this work are evaluated *qualitatively* through visual inspection of the four exported PLY files and side-by-side comparison with the rendered Gazebo scene. We do not report quantitative reconstruction metrics because the simulator does not directly export a per-object ground-truth triangle mesh in the same coordinate frame as the captured

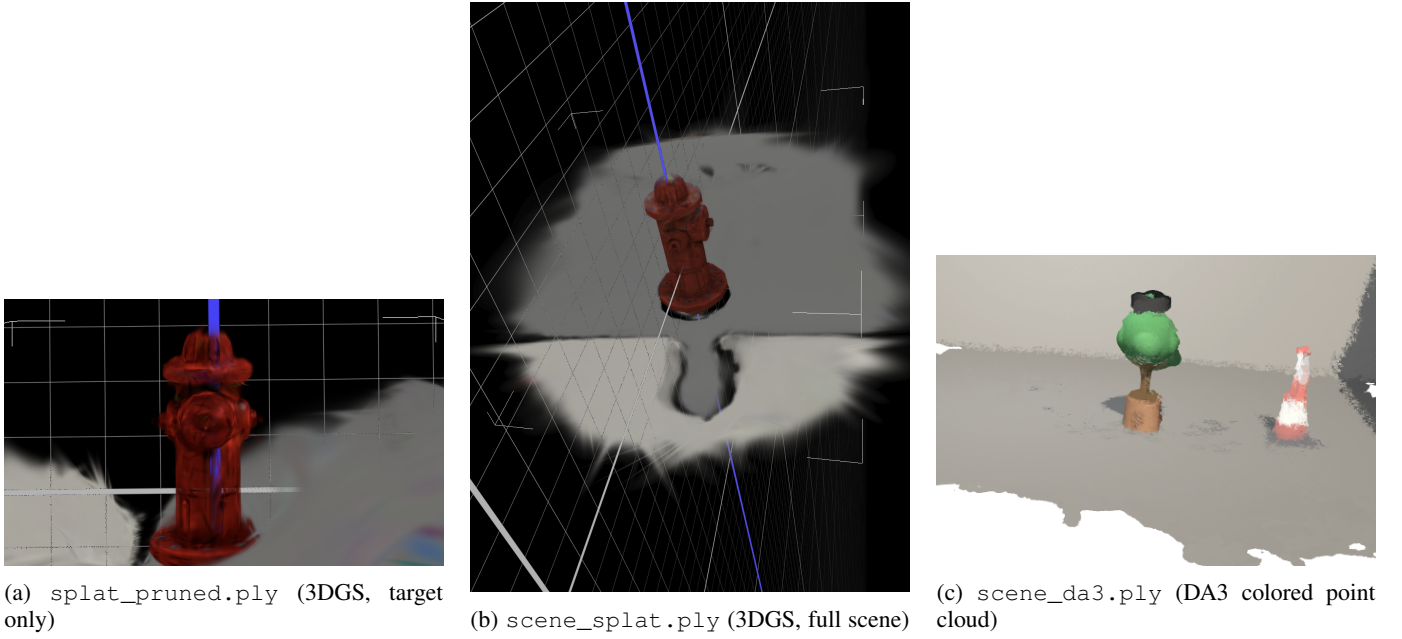


Fig. 4: Representative reconstruction outputs after one mission. The pruned splat (a) and the full-scene splat (b) come from the same splatfacto training run but use different export bounding boxes. The point cloud (c) is the DA3 multi-view depth fusion result, used as input to the Poisson surface mesh.

camera poses, and constructing one would itself introduce alignment error. Future work should add objective metrics including *Chamfer distance* between the DA3 cloud and a sampled ground-truth mesh, *point-to-mesh distance* between the Poisson mesh and the SDF geometry, and *PSNR/SSIM* between renders of the trained 3DGS model and held-out test views captured at unseen orbit angles. We report localization accuracy quantitatively (Table I) because that metric does not require an aligned ground-truth mesh.

E. Live diagnostic view

Fig. 5 shows the custom `live_view` node during `ORBIT_HIGH`. The drone is locked on a relocated potted plant; the lightweight OpenCV tracker has the target boxed in cyan with a label `[TRK:depth] potted plant`. The status bar at the bottom reports the current state, framerate, and source.

F. Empirical photorealism check

The reviewer expressed concern that Gazebo’s rendering quality may not be sufficient for SAM 3. Our results indicate that this concern is manageable in our controlled scene, though not negligible. With the final asset set, observed SAM 3 confidence scores were 0.85 (plant), 0.94 (hydrant), 0.87 (bin), 0.64 (mailbox), and 0.72 (bench), all above the 0.30 detection threshold. However, three Gazebo-Fuel meshes (AirportBench, TrashBin, Mailbox) did *not* register reliably in preliminary tests and were replaced with primitive geometries constructed from boxes and cylinders. This supports the narrower conclusion that Gazebo can be sufficient for this pipeline when visual



Fig. 5: Custom `live_view` window during `ORBIT_HIGH` on the potted plant target. The cyan box is the OpenCV tracker output (used during orbit for speed because SAM 3 is too slow for this state). The status bar reports the state machine phase, FPS, and detection source. The visible scene contains five distractor objects: hydrant, bench, trash bin, mailbox, and a traffic cone.

assets are selected or simplified carefully; it does not imply that arbitrary Gazebo assets will work out of the box.

G. Failure modes and recovery

Fig. 6 shows an early splat output where the target localization triangulation diverged due to insufficient ray spread. The result is a strongly-distorted splat with floater artifacts: the orbit was off-center by approximately 1.5 m, and the captured frames looked at empty space for a substantial fraction of the orbit. We addressed this with the 25° angular-spread gate



Fig. 6: An early failure: the triangulation diverged on near-parallel rays during a fast initial test, producing an off-center orbit and a splat with significant floater artifacts. The 25° angular-spread gate and parallel-ray fallback introduced afterward prevented this failure in subsequent tests.

(Section IV) and the score-weighted cluster selection. After the fix, no subsequent test exhibited this failure mode.

H. Ablation: effect of key design decisions

We did not run a formal ablation study, but the practical effect of three key design choices was directly observed during development:

- **Without the 25° angular-spread gate**, the bearing-only triangulation in (1) consistently diverged when SAM 3 hits arrived from a column of nearly collinear drone positions. The least-squares matrix $\sum_i \mathbf{P}_i$ became ill-conditioned and the estimate drifted to the centroid of the ray origins. We observed orbit-center errors of 1.5–2.0 m before introducing the gate, followed by sub-meter errors in the successful cases after the gate was added.
- **Without the virtual gimbal pitch** (4), the OpenCV tracker reliably drifted during ORBIT_HIGH because the target slid to the bottom edge of the frame and the tracker locked onto whichever distractor object remained near image center, typically a traffic cone in our scene. After introducing the virtual gimbal, the target stayed near image center for the full orbit and tracker drift was not observed in subsequent tests.
- **Without scene randomization**, an implementation bug or convenience shortcut—such as caching an `expected_position` from `targets.py` as a fallback—could silently leak SDF position priors into the runtime path without producing visible failures. By forcing a new permutation at every launch, any such regression would immediately manifest as the drone flying to the wrong place and failing to confirm the target.

TABLE II: Simulator comparison for this project (qualitative)

Requirement	Gazebo	Isaac Sim	AirSim
ROS 2 workflow	first-class	available bridge	wrapper-based
Cost/source model	free/open	closed NVIDIA stack	free/open
Rendering	sufficient here	stronger, GPU-heavy	Unreal-based
Scene authoring	SDF/XML	USD	Unreal project
Deterministic pose control	simple	possible, heavier	possible
8 GB GPU practicality	yes	poor for current 5.1 specs	yes
Maintenance status	active	active	latest official release 2022

I. Comparison to alternative simulators

Table II summarizes why Gazebo was selected for this implementation. The comparison is qualitative and specific to the available hardware and project goals, not a general ranking of simulators.

J. Reproducibility

A full mission is reproducible from a single launch command:

```
ros2 launch drone_recon scenel.launch.py \
  target:="potted plant" seed:=42
```

The `seed:=N` parameter, where `N` is any non-zero integer, makes `scene_randomizer` produce a deterministic permutation, so the same launch with the same seed re-runs the same physical layout. Outputs are written to `~/recon_output/exports/` as four PLY files: `splat_pruned.ply`, `scene_splat.ply`, `scene_da3.ply`, and `scene_mesh.ply`. The captured images and camera-pose JSON are written to `~/recon_output/`, and the per-tick CSV flight log is written to `~/flight_logs/`. SAM 3 debug images for the first ten detections are saved to `/tmp/sam3_dbg/` for visual verification of identification quality. All source code, the world SDF, and the `targets.py` configuration are versioned under `~/ros2_ws/src/drone_recon/`.

X. DISCUSSION

A. Why bearing-only triangulation suits this task

The masked depth measurement carries roughly 30–50 cm RMS noise even after the cleaning steps described above. Multiplying that noisy distance by a ray direction propagates the error into the final XY estimate. The bearing-only formulation in (1) exploits the fact that the mask-centroid direction is much less noisy than the sampled depth value in our setup: the angular error of a SAM 3 mask centroid is small compared with the fractional-meter error observed in depth sampling. Trading noisy distance for cleaner direction is the key insight. The method has been used for decades in bearing-only target tracking [22], sonar, and astronomical observation.

B. Why we did not use SAM 3D

A reviewer suggested SAM 3D from Meta (released 2025), which generates a 3D shape estimate from a single image and segmentation mask. Our use case differs in that the orbit produces multiple calibrated views per target, so multi-view geometry can exploit these additional observations using known simulator poses. SAM 3D’s single-view output would

still be useful as a per-frame shape prior to regularize splatfacto, and we suggest this integration as a future extension.

C. Limitations

We clarify the precise scope of the autonomy claim: the system is fully autonomous with respect to *target discovery*, *mission execution*, and *multi-view image capture*; it is *not* fully autonomous with respect to vehicle self-localization, because the drone reads its own world pose from Gazebo’s ground-truth state rather than estimating it from onboard SLAM. Adding a visual-SLAM front end such as ORB-SLAM3 or VINS-Mono is the natural next step for hardware deployment and would not require changes to the perception, planning, or reconstruction stages of the pipeline.

The pipeline assumes a static, indoor, single-target inspection task. The lawnmower search trajectory does not adapt to obstacles because the simulated room is empty along the search corridors; hardware deployment would need a costmap-based planner. The kinematic pose-control mode improves reconstruction repeatability but is not a physically realistic controller. Reconstruction quality is shown qualitatively rather than with ground-truth geometry metrics; future work should report Chamfer distance, mesh-to-model error, point-cloud completeness, and image-rendering metrics such as PSNR and SSIM where applicable. The bench primitive remains the weakest target for SAM 3 recognition and localization, showing that target geometry and visual distinctiveness still matter. Finally, splatfacto training remains the longest single step at approximately 10–15 minutes per target on an 8 GB GPU; faster 3DGS variants such as 3DGS-MCMC [23] could reduce this.

XI. CONCLUSION

We presented an autonomous simulated UAV target-inspection pipeline that, given only the natural-language name of a target object, visually finds the object under an unknown object-to-slot assignment and produces four 3D representations in approximately 20 minutes per target. The system uses no fiducial markers and no target-position prior, although it does rely on Gazebo’s ground-truth vehicle pose for self-localization in simulation. Scene randomization and the moved-target test support the claim that target discovery is vision-based. The work evolved from an AprilTag / COLMAP proposal into a SAM 3 / DA3 / splatfacto hybrid, addressing the simulator-photorealism concern through careful asset selection and demonstrating reliable SAM 3 inference on the selected Gazebo *ogre2* assets. The bearing-only triangulation primitive, combined with a 25° angular-spread gate, is the central technical contribution: it separates the cleaner angular signal from the noisier depth signal and provides a principled fallback when the geometry is degenerate.

Future work includes replacing kinematic pose control with a PX4 velocity controller for hardware deployment, integrating SAM 3D as a per-frame shape prior to regularize splatfacto, adopting adaptive view planning to reduce the fixed image

budget, and performing a real-drone evaluation in an indoor arena.

ACKNOWLEDGMENT

The author thanks the instructor and TAs of the Intelligent Mobile Robotics class at Binghamton University for project guidance and review feedback that motivated several of the design decisions reported here, particularly the move from fiducial-based detection to open-vocabulary segmentation.

REFERENCES

- [1] E. Olson, “AprilTag: A robust and flexible visual fiducial system,” in *Proc. IEEE Int. Conf. Robotics and Automation*, 2011, pp. 3400–3407.
- [2] J. Wang and E. Olson, “AprilTag 2: Efficient and robust fiducial detection,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems*, 2016, pp. 4193–4198.
- [3] J. L. Schönberger and J. M. Frahm, “Structure-from-motion revisited,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2016, pp. 4104–4113.
- [4] J. L. Schönberger, E. Zheng, M. Pollefeys, and J. M. Frahm, “Pixelwise view selection for unstructured multi-view stereo,” in *Proc. Eur. Conf. Computer Vision*, 2016, pp. 501–518.
- [5] A. Kirillov *et al.*, “Segment anything,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2023, pp. 4015–4026.
- [6] N. Carion *et al.*, “SAM 3: Segment anything with concepts,” *arXiv:2511.16719*, 2025.
- [7] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3D Gaussian splatting for real-time radiance field rendering,” *ACM Trans. Graph.*, vol. 42, no. 4, 2023.
- [8] M. Tancik *et al.*, “Nerfstudio: A modular framework for neural radiance field development,” in *ACM SIGGRAPH 2023 Conf. Proc.*, 2023.
- [9] L. Yang *et al.*, “Depth Anything: Unleashing the power of large-scale unlabeled data,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition*, 2024, pp. 10371–10381.
- [10] H. Lin, S. Chen, J. H. Liew, D. Y. Chen, Z. Li, G. Shi, J. Feng, and B. Kang, “Depth Anything 3: Recovering the visual space from any views,” *arXiv:2511.10647*, 2025.
- [11] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Sci. Robot.*, vol. 7, no. 66, 2022.
- [12] N. Koenig and A. Howard, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems*, 2004, pp. 2149–2154.
- [13] NVIDIA, “Isaac Sim requirements,” *Isaac Sim Documentation*, 2026. [Online]. Available: <https://docs.isaacsim.omniverse.nvidia.com/5.1.0/installation/requirements.html>
- [14] Microsoft, “Releases: microsoft/AirSim,” GitHub repository, latest official release v1.8.1, 2022. [Online]. Available: <https://github.com/microsoft/AirSim/releases>
- [15] B. Mildenhall *et al.*, “NeRF: Representing scenes as neural radiance fields for view synthesis,” in *Proc. Eur. Conf. Computer Vision*, 2020, pp. 405–421.
- [16] M. Kazhdan, M. Bolitho, and H. Hoppe, “Poisson surface reconstruction,” in *Proc. Eurographics Symp. Geometry Processing*, 2006, pp. 61–70.
- [17] Q.-Y. Zhou, J. Park, and V. Koltun, “Open3D: A modern library for 3D data processing,” *arXiv:1801.09847*, 2018.
- [18] A. Radford *et al.*, “Robust speech recognition via large-scale weak supervision,” in *Int. Conf. Machine Learning*, 2023.
- [19] A. Yang *et al.*, “Qwen2.5 technical report,” *arXiv:2412.15115*, 2024.
- [20] S. Zhang, C. Liu, and N. Haala, “Three-dimensional path planning of UAVs imaging for complete photogrammetric reconstruction,” *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. V-1-2020, pp. 325–331, 2020.
- [21] N. Smith, N. Moehle, M. Goesele, and W. Heidrich, “Aerial path planning for urban scene reconstruction: A continuous optimization method and benchmark,” *ACM Trans. Graph.*, vol. 37, no. 6, 2018.
- [22] S. C. Nardone and V. J. Aidala, “Observability criteria for bearings-only target motion analysis,” *IEEE Trans. Aerospace and Electronic Systems*, vol. AES-17, no. 2, pp. 162–166, 1981.

- [23] S. Kheradmand *et al.*, “3D Gaussian splatting as Markov chain Monte Carlo,” in *Advances in Neural Information Processing Systems*, 2024.
- [24] M. Gallai, “Autonomous sim drone for target orbit imaging and 3D reconstruction,” Project Proposal, Intelligent Mobile Robotics, Binghamton University, March 2026.